Universidad Nacional de San Martín

TP Integrador Base de Datos

Materia	Base de Datos
Carrera	Tecnicatura en Programación Informática
Docentes	Gabriel FLORES - Rodrigo CASCO
Proyecto	Alero: sistema de reservas y reseñas para alojamientos
Integrantes	(completar)
Fecha de entrega	6 de noviembre de 2026
Repositorio	https://github.com/ju4nchp/tp-integrador-bd-alero
Aplicación publicada	https://alero-tp-bd.onrender.com

1. Descripción del problema

Elegimos el dominio de las reservas de viajes. Un alojamiento (hotel, hostel o cabañas) recibe reservas, algunas se cancelan y, cuando la estadía termina, el huésped puede dejar una reseña. El sistema lo usa el dueño o encargado del alojamiento para cargar, consultar, modificar y dar de baja reservas, cancelarlas con un motivo, cargar y responder reseñas y ver estadísticas de facturación y cancelaciones.

Definimos tres reglas: las reservas no se borran (la baja es lógica y quedan en un historial), solo se modifican o cancelan las pendientes o confirmadas, y las reseñas son solo de estadías completadas, una por reserva, y no se editan ni se borran.

2. Descripción y origen del dataset

Usamos el archivo **Travel_And_Tourism.csv**. Origen: (completar con el link de donde lo descargamos). Tiene 1000 filas y 29 columnas: cada fila es una reserva y trae todo junto (cliente, destino, hotel, viaje, pago, estado y, si corresponde, la reseña o el motivo de cancelación). Hay 1000 clientes, 78 hoteles, 26 ciudades de destino en 10 países y 44 ciudades en total. De las reservas, 772 están completadas, 141 canceladas, 79 confirmadas y 8 pendientes.

Grupo	Columnas del CSV
Reserva	Booking_ID, Booking_Date, Travel_Date, Return_Date, Booking_Status
Cliente	Customer_ID, Customer_Name, Customer_Age, Gender, Customer_City, Customer_Country
Destino y hotel	Destination_City, Destination_Country, Hotel_Name, Hotel_Rating
Viaje	Number_of_Travellers, Number_of_Adults, Number_of_Children, Number_of_Rooms, Number_of_Nights, Meal_Plan, Transportation_Type
Pago	Payment_Method, Discount_Amount, Total_Trip_Cost
Cancelación y reseña	Cancellation_Status, Cancellation_Reason, Customer_Rating, Customer_Review_Comment

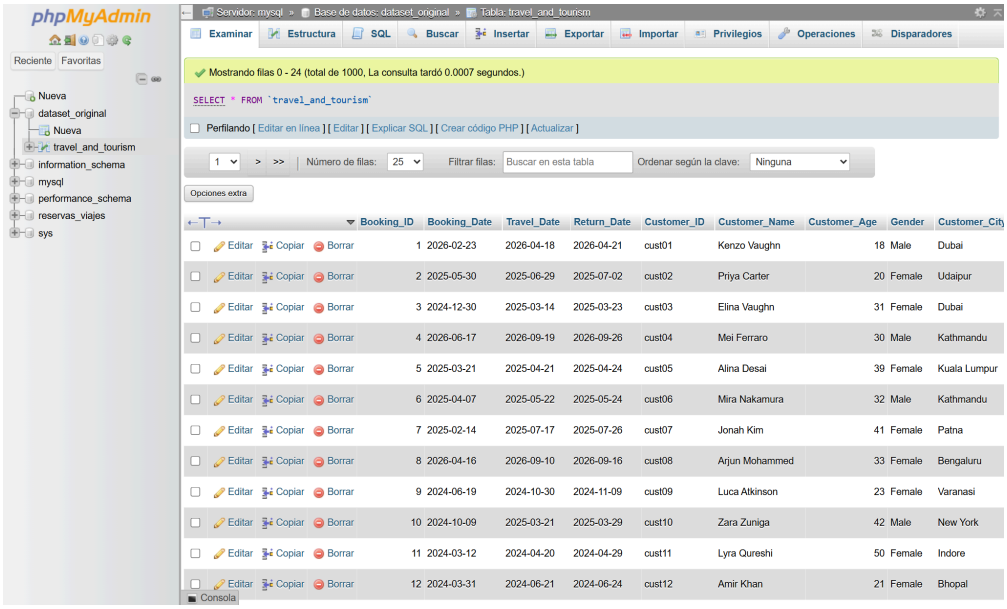


Figura 1. El CSV cargado tal cual en MySQL: una sola tabla de 29 columnas.

3. Análisis inicial de los datos

Cargamos el CSV en una tabla de MySQL (script database/analisis_dataset.sql) y lo revisamos con consultas en phpMyAdmin. Encontramos:

- **Datos repetidos:** el nombre, la ciudad y el país del hotel se repiten en cada reserva (Cedar Heights aparece en 25 filas). Lo mismo pasa con la ciudad y el país del cliente.
- **Dependencias:** Customer_ID determina nombre, edad, género y ciudad; el hotel determina su ciudad, y la ciudad determina el país (ninguna ciudad tiene más de un país, Figura 2). Las dos últimas son dependencias transitivas.
- **Columnas que se calculan:** viajeros = adultos + niños, noches = regreso − salida, y el estado de cancelación sale del estado de la reserva. Coinciden en las 1000 filas (Figura 3).
- **Valores vacíos disfrazados:** 859 motivos de cancelación dicen "Not Applicable" y 228 puntajes valen 0 porque no hay reseña. La cancelación y la reseña son datos opcionales.
- **Hotel_Rating** cambia en cada reserva del mismo hotel (de 1.0 a 5.0): es el puntaje del cliente, no un dato fijo del hotel.

Con todo en una tabla hay problemas de **actualización** (cambiar el nombre de un hotel obliga a tocar todas sus filas), de **inserción** (no se puede cargar un hotel sin una reserva) y de **eliminación** (si se borra la única reserva de un cliente se pierden sus datos; en el CSV cada cliente tiene una sola reserva).

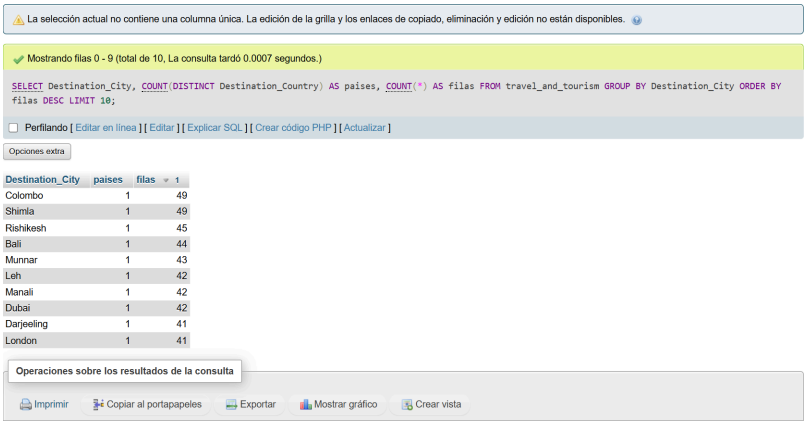


Figura 2. Cada ciudad tiene un solo país (columna paises en 1).

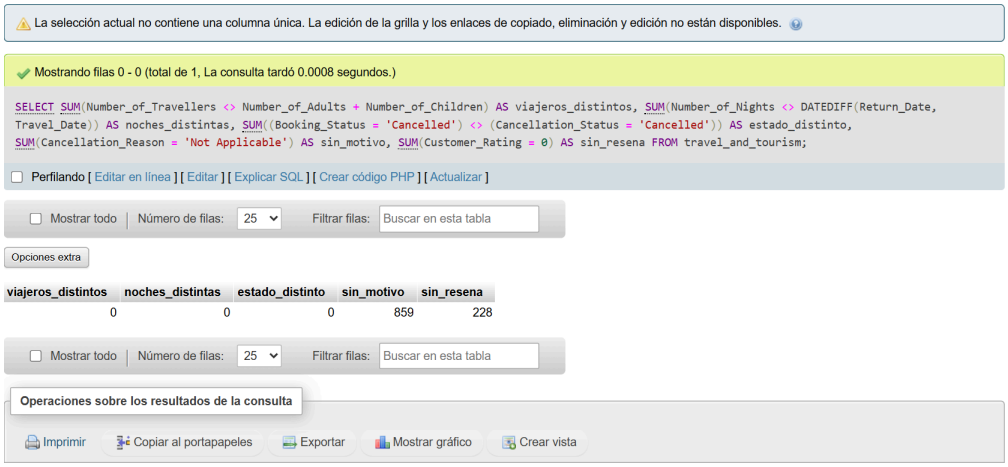


Figura 3. Las columnas calculadas no tienen diferencias, y hay 859 "Not Applicable" y 228 puntajes en 0.

4. Proceso de normalización

Paso	Cómo quedó
Origen	1 tabla de 29 columnas, una fila por reserva.
1FN	Ya cumple: valores atómicos, sin grupos repetidos y Booking_ID único. Las etiquetas de las reseñas (una lista) van en su propia tabla, resena_etiqueta, para no romper la 1FN.
2FN	Ya cumple: la clave es una sola columna, así que no hay dependencias parciales.
3FN	No cumple por las dependencias transitivas: separamos país, ciudad, cliente y hotel.
Resultado	12 tablas con PK, FK y restricciones CHECK.

Las dependencias transitivas que eliminamos son:

```
Booking_ID → Customer_ID → Customer_Name, Customer_Age, Gender, Customer_City →
Customer_Country
Booking_ID → Hotel_Name → Destination_City → Destination_Country
```

- **PAIS** (nombre, region), **CIUDAD** (id, nombre, pais_nombre), **CLIENTE** y **HOTEL** quedan en tablas propias. Una sola tabla CIUDAD sirve para clientes y hoteles. La región no venía en el CSV y la agregamos para agrupar estadísticas.
- Meal_Plan, Payment_Method, Transportation_Type y Booking_Status pasan a catálogos (regimen, medio_pago, transporte y estado_reserva) para no repetir texto.
- Sacamos Number_of_Travellers, Number_of_Nights y Cancellation_Status porque se calculan.
- **CANCELACION** existe solo para las 141 canceladas y **RESENA** solo para las 772 completadas, así no se guardan "Not Applicable" ni puntajes en 0. Hotel_Rating queda como puntaje de la reseña, y el hotel guarda una categoría base con el promedio.

5. Diagrama Entidad-Relación

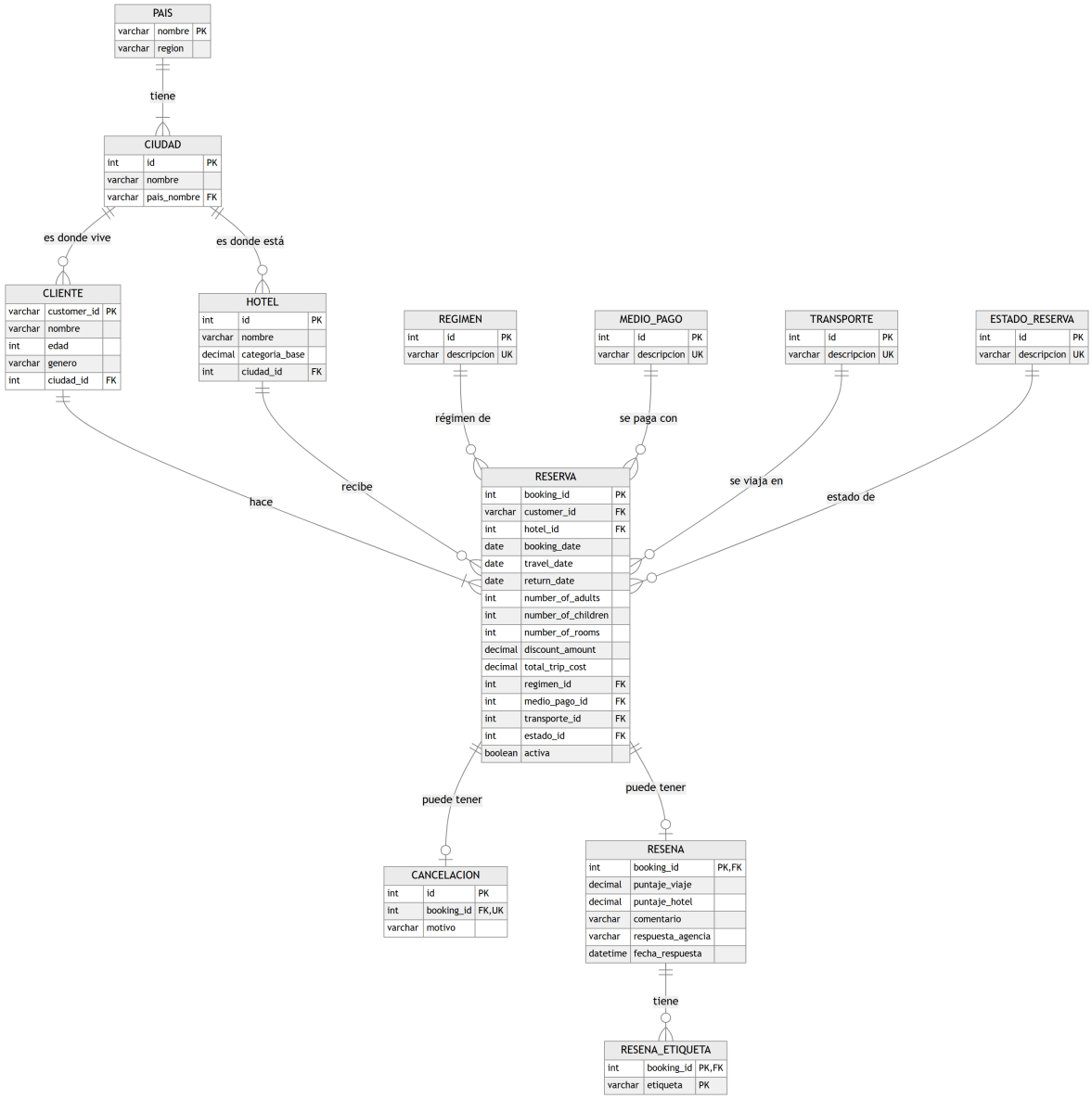


Figura 4. DER de la base reservas_viajes (también en documentacion/DER.png).

Un país tiene una o más ciudades, y en cada ciudad viven clientes y hay hoteles (1:N). RESERVA resuelve la relación N:M entre CLIENTE y HOTEL, y cada reserva tiene un régimen, un medio de pago, un transporte y un estado (N:1). Una reserva puede tener una cancelación y una reseña, o ninguna (1:0..1), y una reseña puede tener varias etiquetas (1:N).

6. Modelo relacional

database/modelo_relacional.sql crea la base con las 12 tablas y database/datos_iniciales.sql carga los datos. Este último lo genera nuestro script database/cargar_datos.py a partir del CSV, después de verificar las dependencias y las reglas.

Tabla	Columnas	PK / FK	Restricciones
pais	nombre VARCHAR(100), region VARCHAR(100)	PK nombre	region NOT NULL
ciudad	id INT, nombre VARCHAR(100), pais_nombre VARCHAR(100)	PK id; FK pais_nombre → pais	UNIQUE (nombre, pais_nombre)
cliente	customer_id VARCHAR(50), nombre VARCHAR(150), edad INT, genero VARCHAR(20), ciudad_id INT	PK customer_id; FK ciudad_id → ciudad	edad ≥ 18; genero 'Male' o 'Female'
hotel	id INT, nombre VARCHAR(150), categoria_base DECIMAL(3,1), ciudad_id INT	PK id; FK ciudad_id → ciudad	UNIQUE (nombre, ciudad_id); categoría entre 1 y 5
regimen, medio_pago, transporte, estado_reserva	id INT, descripcion VARCHAR(100)	PK id	descripcion UNIQUE
reserva	booking_id INT, customer_id, hotel_id, booking_date, travel_date, return_date DATE, number_of_adults, number_of_children, number_of_rooms INT, discount_amount, total_trip_cost DECIMAL(10,2), regimen_id, medio_pago_id, transporte_id, estado_id INT, activa BOOLEAN	PK booking_id; FK a cliente, hotel y los 4 catálogos	reserva ≤ salida < regreso; adultos ≥ 1; niños ≥ 0; habitaciones entre 1 y los viajeros; costo > 0; descuento entre 0 y el costo
cancelacion	id INT, booking_id INT, motivo VARCHAR(255)	PK id; FK booking_id → reserva	booking_id UNIQUE
resena	booking_id INT, puntaje_viaje, puntaje_hotel DECIMAL(2,1), comentario, respuesta_agencia VARCHAR, fecha_respuesta DATETIME	PK y FK booking_id → reserva	puntajes entre 1 y 5
resena_etiqueta	booking_id INT, etiqueta VARCHAR(40)	PK (booking_id, etiqueta); FK → resena	-

Las FK usan RESTRICT: no se puede borrar una fila que otra tabla usa. Probamos que las restricciones funcionan: MySQL rechaza una reserva que vuelve antes de salir (Figura 6) y no deja borrar una reserva con cancelación (Figura 7).

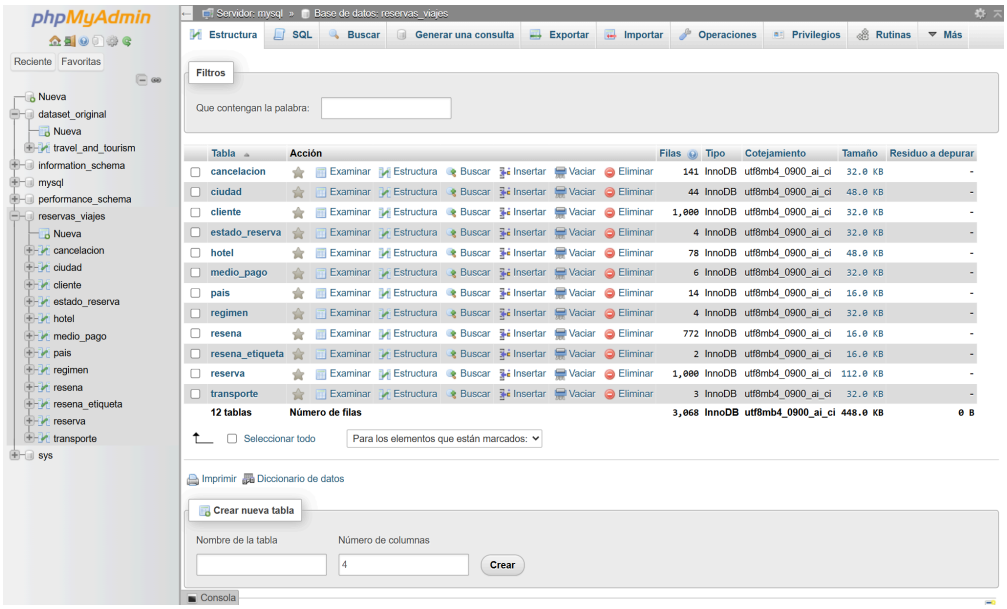


Figura 5. Las 12 tablas en phpMyAdmin con los datos cargados.

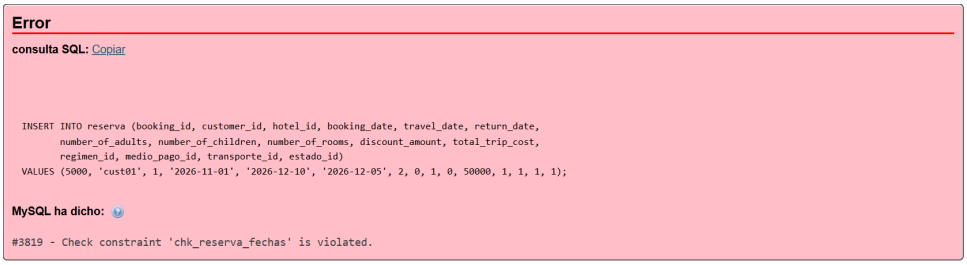


Figura 6. La restricción chk_reserva_fechas rechaza la reserva.

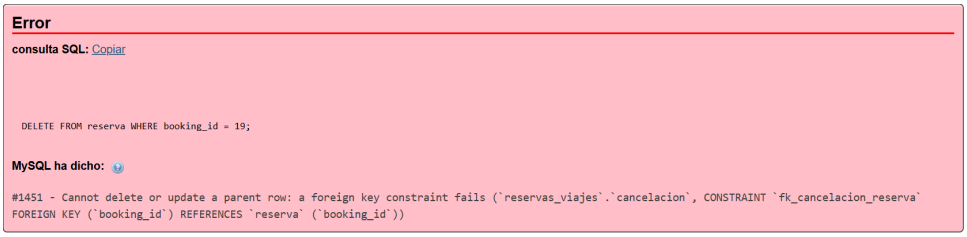


Figura 7. La clave foránea de cancelacion impide borrar la reserva 19.

7. Diseño de la solución NoSQL

Usamos **MongoDB**, una base documental, con dos colecciones que genera el mismo script de carga y se importan con mongoimport:

- **reservas:** un documento por reserva, con el cliente, el hotel y las fechas adentro, los catálogos como texto y la cancelación como subdocumento solo cuando existe (Figura 8).
- **resenas:** un documento por reseña, con el hotel copiado, los puntajes, el comentario, un array de etiquetas y un objeto opcional con la respuesta del alojamiento (Figura 9).

Como MongoDB no tiene FK ni CHECK, el script indices.js agrega validadores $jsonSchema (campos obligatorios, tipos, estados permitidos, edad mínima y puntajes entre 1 y 5; Figura 10) e

índices en estado, hotel.id y hotel.pais, y un índice único en reservaId (una reseña por reserva). El backend controla además que la reserva exista y esté completada antes de guardar una reseña.

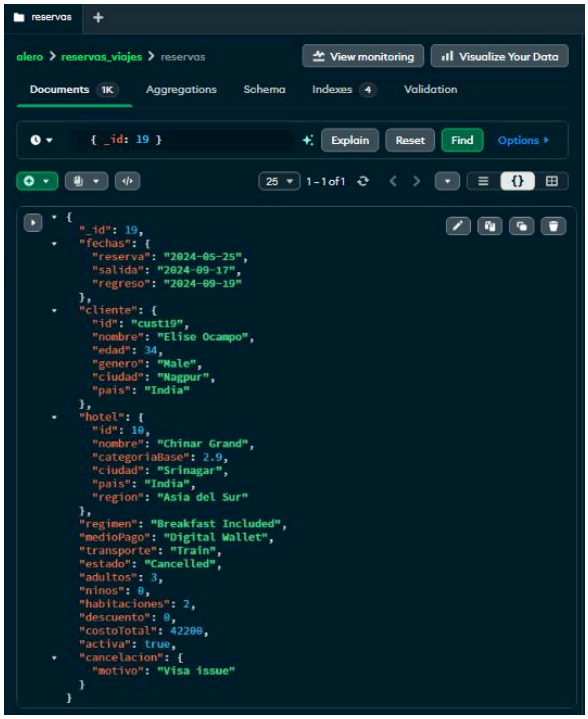


Figura 8. La reserva 19 en MongoDB Atlas.

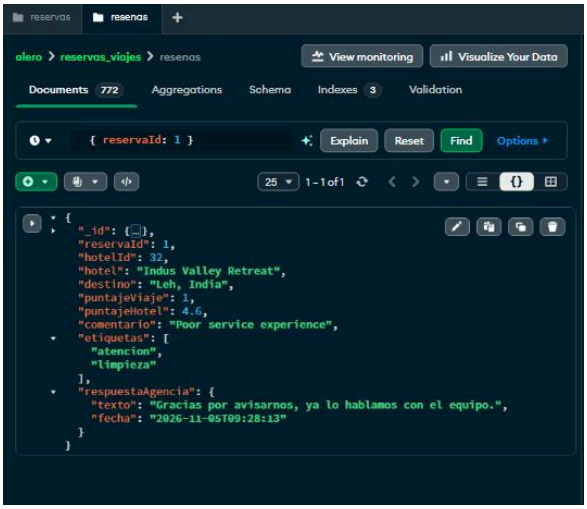


Figura 9. Una reseña con etiquetas y respuesta.

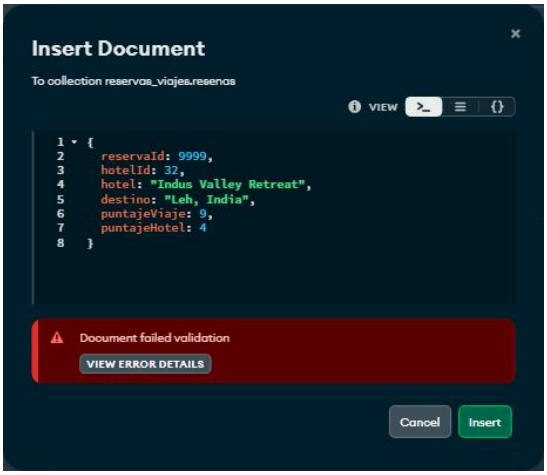


Figura 10. El validador rechaza un puntaje de 9.

8. Justificación de la utilización de NoSQL

La parte del sistema que más se beneficia de NoSQL son las **reseñas**: las etiquetas son una lista de largo variable (un array en MongoDB, una tabla aparte y un JOIN en MySQL) y la respuesta es opcional (un objeto que aparece solo si existe, en vez de columnas en NULL). La consulta más común, las reseñas de un alojamiento, se resuelve con la colección sola. También conviene tener la **reserva completa** en un documento, que se lee sin los 9 JOIN que necesita MySQL. Elegimos una base documental porque los datos se usan como fichas completas.

- **Ventajas:** lectura en un solo acceso, esquema flexible para datos opcionales y listas, y estadísticas con un aggregate sobre una colección.
- **Desventajas:** datos duplicados (si un hotel cambia de nombre hay que actualizar muchos documentos, la anomalía que la 3FN evita), integridad que depende de validadores y del backend, y menos garantías que una transacción entre tablas.

Cómo conviven los dos motores: funcionan al mismo tiempo detrás del mismo backend, cada uno con su copia de los datos cargada desde el mismo CSV, y la aplicación puede trabajar con cualquiera de los dos. Así se cumple el backend polimórfico y se puede comparar cómo se resuelve lo mismo en cada modelo, con estructuras distintas: 12 tablas y 2 colecciones.

9. Arquitectura general de la aplicación

Frontend (React) → Backend (Kotlin + Spring Boot) → capa de persistencia (ReservaRepository, ResenaRepository, CatalogoRepository) → implementación MySQL o implementación MongoDB. El frontend nunca accede a las bases. Todo se levanta con Docker Compose, y hay una versión en la nube con planes gratuitos (Render, Aiven para MySQL y MongoDB Atlas).

10. Explicación del backend polimórfico

Cada repositorio es una interfaz con dos implementaciones: MySqlReservaRepository (Spring Data JPA) y MongoReservaRepository (MongoTemplate), y lo mismo para reseñas y catálogos. Los servicios y las reglas de negocio usan solo la interfaz. El frontend manda el header X-DB-Engine (mysql o mongo) según el selector de la pantalla; sin header se usa MySQL. Con los dos motores se puede dar de alta, consultar, listar, modificar, dar de baja y cancelar reservas, cargar y responder reseñas, y correr las estadísticas por país y por alojamiento.

11. Tecnologías utilizadas

MySQL 8.4 con phpMyAdmin, MongoDB 7 (MongoDB Atlas en la nube), Python con pandas para la carga, Kotlin con Spring Boot para el backend, React con Vite y TypeScript para el frontend, Docker Compose, Git y GitHub.

12. Instrucciones para ejecutar el proyecto

Con Docker abierto, en la carpeta del proyecto: `docker compose up --build`. Eso carga las dos bases y levanta la aplicación en http://localhost:5173 (phpMyAdmin en http://localhost:8081). También se puede usar sin instalar nada en https://alero-tp-bd.onrender.com. El README.md tiene el detalle.

13. Ejemplos de funcionamiento

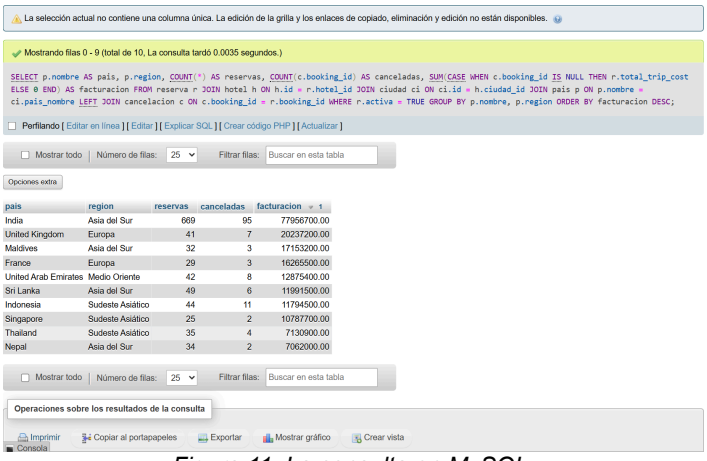
Consulta compleja: facturación y tasa de cancelación por país de destino. En MySQL:

```
SELECT p.nombre AS pais, p.region, COUNT(*) AS reservas, COUNT(c.booking_id) AS canceladas,
       SUM(CASE WHEN c.booking_id IS NULL THEN r.total_trip_cost ELSE 0 END) AS facturacion
  FROM reserva r
  JOIN hotel h ON h.id = r.hotel_id
  JOIN ciudad ci ON ci.id = h.ciudad_id
  JOIN pais p ON p.nombre = ci.pais_nombre
  LEFT JOIN cancelacion c ON c.booking_id = r.booking_id
```

```
WHERE r.activa = TRUE
GROUP BY p.nombre, p.region ORDER BY facturacion DESC;
```

En MongoDB, sin JOIN, porque el país del hotel está en el documento:

```
db.reservas.aggregate([
  { $match: { activa: true } },
  { $group: { _id: "$hotel.pais", reservas: { $sum: 1 },
    canceladas: { $sum: { $cond: [{ $eq: ["$estado", "Cancelled"] }, 1, 0] } },
    facturacion: { $sum: { $cond: [{ $eq: ["$estado", "Cancelled"] }, 0, "$costoTotal"] } } } },
  { $sort: { facturacion: -1 } }
])
```



pais	region	reservas	canceladas	facturacion
India	Asia del Sur	669	95	77956700.00
United Kingdom	Europa	41	7	20237200.00
Maldives	Asia del Sur	32	3	17153200.00
France	Europa	29	3	16265500.00
United Arab Emirates	Medio Oriente	42	8	12875400.00
Sri Lanka	Asia del Sur	49	6	11991500.00
Indonesia	Sudeste Asiático	44	11	11794500.00
Singapore	Sudeste Asiático	25	2	10787700.00
Thailand	Sudeste Asiático	35	4	7130900.00
Nepal	Asia del Sur	34	2	7062000.00

Figura 11. La consulta en MySQL.

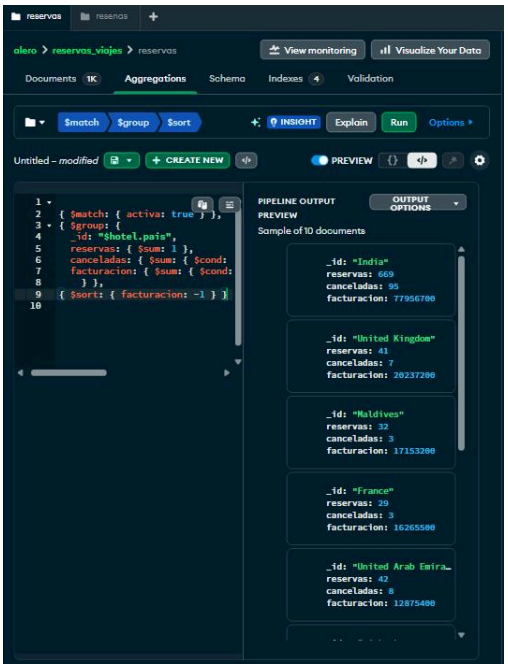


Figura 12. La misma consulta en Atlas.

Las dos dan los mismos números (India: 669 reservas, 95 canceladas y 77.956.700 de facturación). En la aplicación, el botón "Comparar los dos motores" corre las consultas en los dos y lo verifica (Figura 13).



Figura 13. Estadísticas con MongoDB y la comparación de los dos motores.

14. Comparación entre ambas implementaciones

Aspecto	MySQL	MongoDB
Modelo	12 tablas en 3FN	2 colecciones con datos embebidos
Leer una reserva	9 JOIN	un documento
Integridad	PK, FK, UNIQUE y CHECK	validadores, índice único y backend
Etiquetas y respuesta	tabla aparte y columnas en NULL	array y objeto opcional
Datos repetidos	ninguno	cliente y hotel en cada documento
Listado de 20 reservas	22 a 26 ms	10 a 11 ms
Detalle de una reserva	14 a 17 ms	6 ms
Estadística por país	8 ms	8 ms

Los tiempos los medimos con las dos bases en la misma computadora. Con 1000 reservas las diferencias son de milisegundos, pero MongoDB lee más rápido donde MySQL necesita varios JOIN. A cambio, MySQL garantiza la integridad en la propia base.

15. Problemas encontrados y decisiones tomadas

- El puntaje del hotel cambia en cada reserva: lo guardamos en la reseña y el hotel usa el promedio.
- "Not Applicable" y puntajes en 0: los reemplazamos por las tablas opcionales cancelacion y resena.
- La región no venía en el CSV: la agregamos con seis regiones.
- Respecto de la primera versión del modelo agregamos adultos y niños (para calcular los viajeros), transporte_id, activa (baja lógica), resena y resena_etiqueta. El motivo de cancelación quedó como texto y no como catálogo.
- En la propuesta la reseña iba dentro de la reserva en MongoDB; la pasamos a su propia colección porque se carga y se responde en otro momento.

- Primero pensamos tener las reservas solo en MySQL y las reseñas solo en MongoDB. Lo cambiamos para cumplir con el backend polimórfico y poder comparar los dos motores con las mismas consultas.
- En MongoDB el validador no aceptaba exclusiveMinimum: 0; en $jsonSchema va minimum: 0 junto con exclusiveMinimum: true.

16. Conclusiones

Normalizar sacó las repeticiones y las anomalías, y con PK, FK y CHECK la base misma rechaza datos inválidos. El costo son los JOIN para armar una reserva completa, que MongoDB devuelve en un solo documento. MongoDB se adapta mejor a datos opcionales y listas como las reseñas, pero la integridad depende de los validadores y del backend. Las dos implementaciones dieron los mismos números, lo que nos sirvió para validar la carga. No hay un motor mejor en general: el relacional para el núcleo de reservas y el documental para las reseñas, y con un backend con interfaces comunes pueden convivir en la misma aplicación.